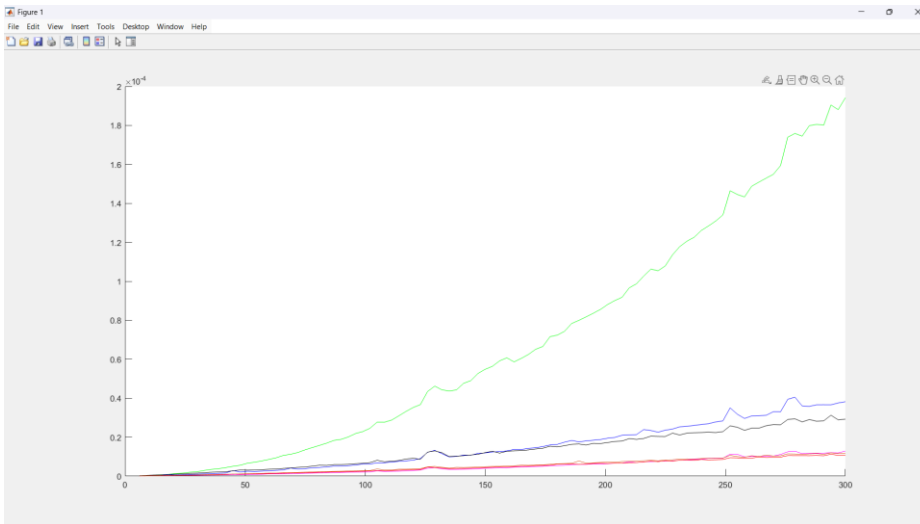


Tempi measure sorts

Il tempo medio per ordinare 300 vettori è riportato in funzione della loro dimensione nel seguente grafico.

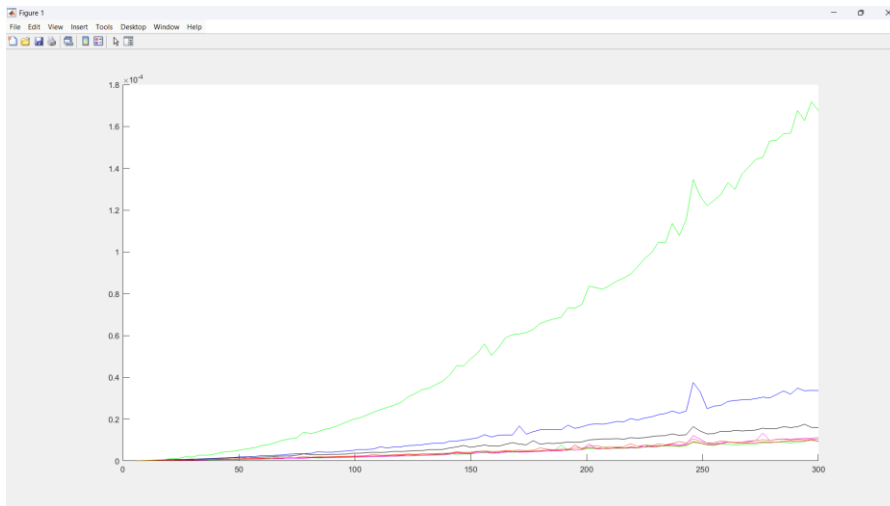


Legenda:

- insertion sort: magenta
- quicksort: arancione
- `std::sort()`: rosso
- bubblesort: verde
- selection sort: blu
- mergesort: nero

qui si osserva che il mergesort è meno efficiente dell'insertion sort per vettori di dimensione < 300 . Per questo motivo, ho modificato l'algoritmo e ho costruito il `mergestort2`. Quest'ultimo, invece di definire due nuovi vettori L ed R ad ogni chiamata del merge, crea un solo vettore v della dimensione di A (il vettore da ordinare), verifica le uguaglianze direttamente su A e poi inserisce in v gli elementi ordinati. Successivamente, copia in A gli elementi di v che sono stati modificati.

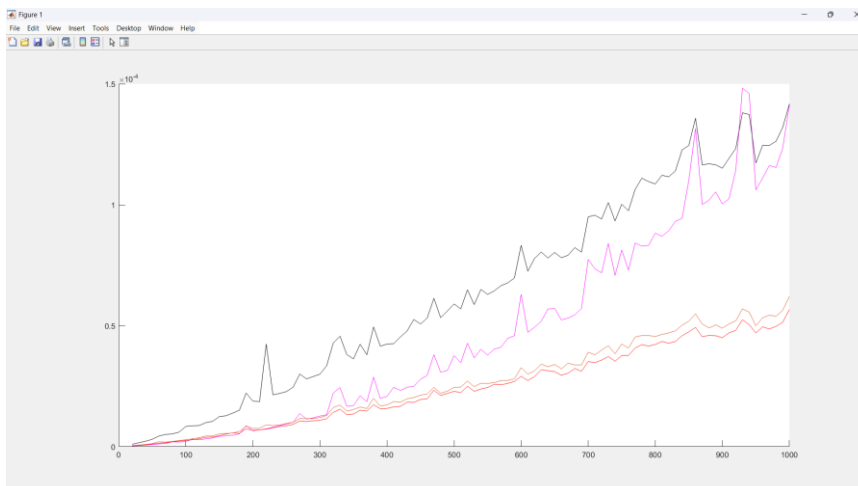
utilizzando questo algoritmo, il mergesort migliora la sua efficienza:



Legenda:

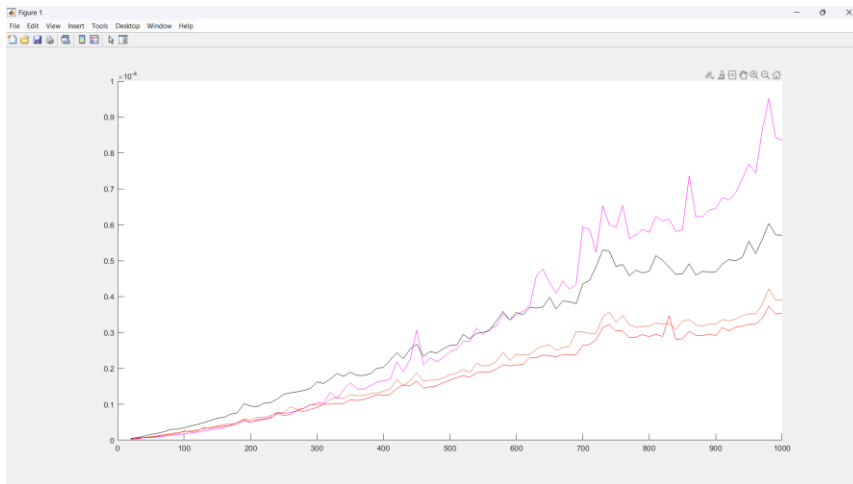
- insertion sort: magenta
- quicksort: arancione
- `std::sort()`: rosso
- bubblesort: verde
- selection sort: blu
- **mergesort2: nero**

Su vettori di lunghezza fino a 1000, la differenza è ancora più marcata:



Legenda:

- insertion sort: magenta
- quicksort: arancione
- `std::sort()`: rosso
- **mergesort: nero**

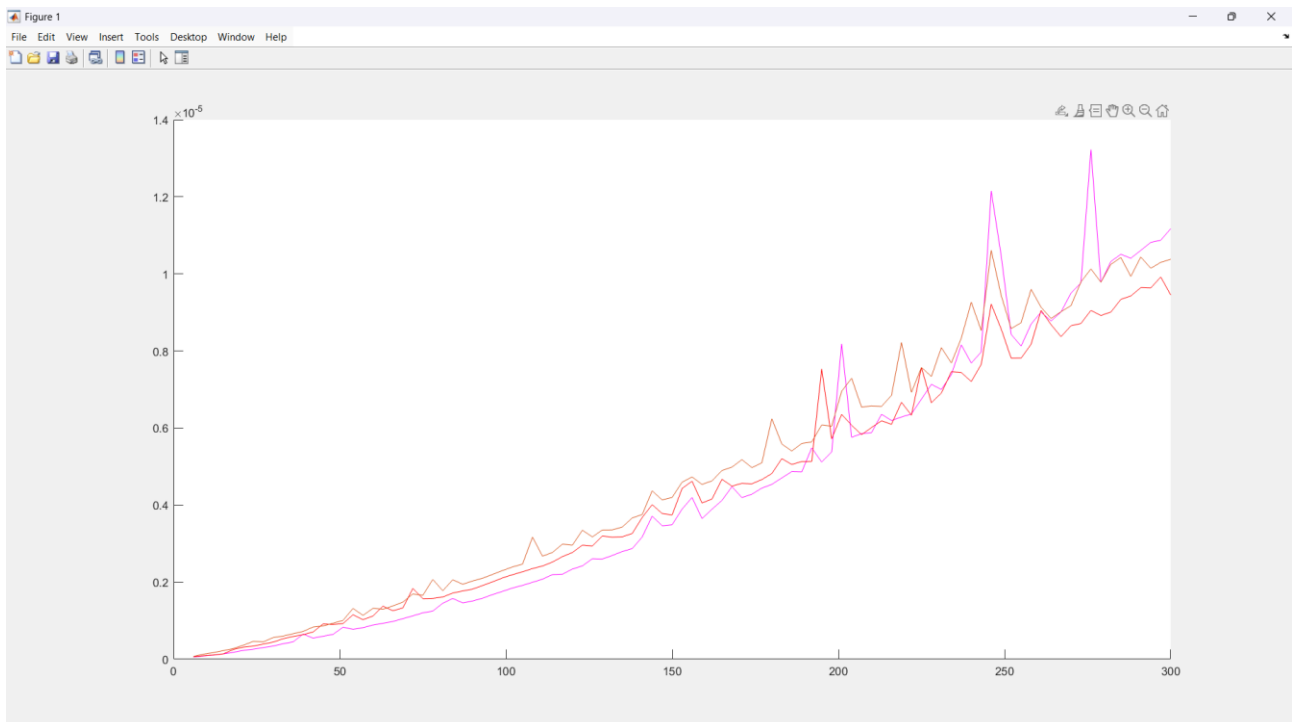


Legenda:

- insertion sort: magenta
- quicksort: arancione
- std::sort(): rosso
- **mergesort2: nero**

È chiaro, tuttavia, che il quicksort (in arancione) sia molto più rapido. Inoltre, osserviamo che la sua performance, per vettori di grandi dimensioni, è simile a quella dello std::sort().

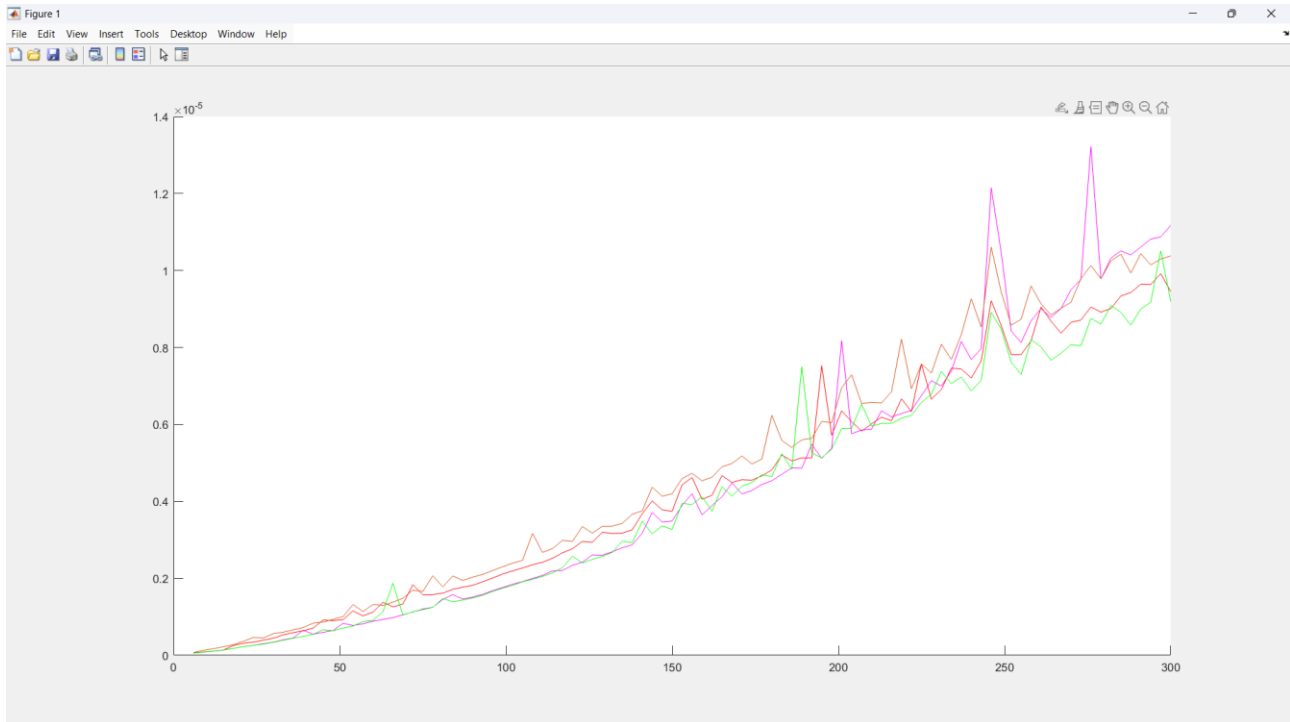
Valutiamo, ora, la lunghezza minima dei vettori per cui il quicksort diventa più efficiente dell'insertion sort:



da questo grafico osserviamo che il quicksort (arancione) diventa più efficiente dell'insertion sort (magenta) solo a partire dimensioni comprese tra i 200 e i 250. Inoltre, lo std::sort() è meno efficiente dell'insertion sort per vettori di lunghezza circa <200.

Si consideri che, rispetto all'esercitazione 4, i tempi di esecuzione dell'insertion sort sono più brevi, poiché l'ho implementato passando il vettore per riferimento e l'ho compilato in modalità release.

Alla luce di questi risultati, ho costruito il programma di ordinamento "personalizzato" (mix_sort) utilizzando l'insertion sort fino a vettori di lunghezza 215 e il quicksort successivamente. Ordinando vettori di lunghezza fino a 300 in modalità release, ho ottenuto il seguente grafico:



Legenda:

- insertion sort: magenta
- quicksort: arancione
- std::sort(): rosso
- mix_sort: verde

osserviamo che il mix sort impiega lo stesso tempo dell'insertion sort fino a vettori di lunghezza 215 (la soglia che ho impostato nel codice). In questa regione si mantiene più veloce anche dello std::sort().

Successivamente, diventa più veloce dello std::sort() ed anche del quicksort stesso. Quest'ultimo risultato è piuttosto inatteso, poiché il mix_sort utilizza il quicksort e, inoltre, controlla la dimensione del vettore, pertanto compie il numero di operazioni del quicksort +1. Il risultato non è imputabile all'inesattezza della costruzione del mix_sort, poiché è stato testato mediante la funzione is.sorted() presente in ordinamento.h.

USO DELL'AI

Ho riscontrato la presenza di termini negativi nei tempi misurati. Dopo aver controllato varie volte il codice, ho cercato quale potesse essere il problema e l'AI mi ha suggerito di sostituire, all'interno di `timecounter.h` e `timecounter.cpp`, `high_resolution_clock` con `steady_clock`. Con questa modifica non ho riscontrato più termini negativi nella misurazione dei tempi.